Stage 4 Question Script

What to say when Dr. Cheng asks about the code, and which code to open while you say it.

How to use this during the talk: before the talk, open this file on GitHub in a fourth browser tab. When a question comes in, find its type below, say the line in the quote block, then click **Show** to open the code with the right lines highlighted. Every link points to the version of the code we are demoing, so the line numbers will not move.

Lines in quote blocks are said out loud. **Show** opens the code. **Point at** is what to put your cursor on.

Quick Find

If she asks about...	Go to
IDs, duplicates, "like a car VIN"	1. Keys and duplicates
How the code stops bad data	2. How the code enforces a rule
Where insert, update or delete happens	3. Insert, update and delete
The search or the join	4. Search queries
SQL injection, security, other farms' data	5. Security
Where the data came from, bad rows	6. The data
The advanced functions	7. Advanced functions

1. Keys and Duplicates

"What ID do you use so a record can't be duplicated?"

> "Every table has two layers. First, the database assigns each row its own ID, and nobody can type one in or reuse one. Second, each table has a UNIQUE rule on its real-world key. That second rule is the VIN. For a plot it's the farm plus the plot name, so one farm can't have two fields called North field."

Show: plot table
Point at: `GENERATED ALWAYS AS IDENTITY PRIMARY KEY` on line 113, then `UNIQUE (farm_id, name)` on line 118.

"What is the VIN for the other tables?"

> "It depends on what makes the thing real. A crop in a farm's catalog is one farm plus one plant, so that pair is the primary key. A plant in the taxonomy is its full path from the kingdom down. A product is its slug. A farm can't share a web domain or an owner email with another farm."

Table	What can't repeat	Show
crop_catalog	one farm + one plant	schema.sql lines 121 to 130
taxon	the full path, like Plantae/.../Allium cepa	schema.sql lines 28 to 37
product	the slug, always lowercase	schema.sql lines 40 to 45
product_alias	the cleaned auction label, so one label means one product	schema.sql lines 48 to 54
farm	web domain, owner email	schema.sql lines 101 to 109

"Can I see it happen?"

Go to the demo farm, click **Add a plot**, and type `North field` again (do this before Step 6 deletes it).

> "The database rejects it, and the app tells the user why."

"What stops a harvest being entered twice?"

> "Nothing, on purpose. A grower can pick the same crop twice in one day, so two identical harvests can both be real. Each one gets its own event ID, and every edit or delete is limited to that farm."

Show: farm_event table

"What about the auction prices?" (only if she pushes)

> "Each price row has its own ID, but we don't yet block a repeated report. USDA sometimes reissued the same daily report under a new file name, and our loader loaded each copy. That's 155 of 33,667 rows, half a percent. The fix is a UNIQUE rule on the source, date, label, unit, quantity and prices, and we're adding it for Stage 5."

Show: market_lot table. Point out that it has CHECK rules but no UNIQUE.

2. How the Code Enforces a Rule

"How does your code prevent that?"

> "The rule lives in the database, not in the Python. So every page and every script is held to it, not just one form. The Python only does two things: it rolls back the failed write, and it turns the database error into a sentence the user can read."

Walk through these three links in order:

1. "Here's the rule, in the schema."

Show: plot_name_unique

1. "Every write goes through this one function. If Postgres rejects it, we roll back and pass the error up."

Show: execute() in db.py

1. "Then we read the name of the rule that was broken and look up a plain message for it."

Show: errors.py
Point at: `"plot_name_unique": "This farm already has a plot with that name."`

"What about adding the same crop twice?"

> "That one we handle before it errors. The insert says ON CONFLICT DO NOTHING, so a repeat changes nothing, and the page says the crop is already in the catalog."

Show: INSERT_ENTRY_SQL and catalog_add()

"Where does the 'needs a unit' message from the demo come from?"

> "A named CHECK constraint: if there's a quantity, there has to be a unit."

Show: farm_event_unit_required

"Do you check the form at all before the database?"

> "Only for type and length: a number has to be a number and a name can't be too long. The real rules stay in the database so there's one place to change them."

Show: forms.py

3. Insert, Update and Delete

"Show me the insert / update / delete code."

> "Each table the user changes has its own file. The SQL is written by hand at the top of the file, and

> the page functions below it run that SQL."

Table	SQL	Page code
farm_event	INSERT, UPDATE, DELETE	event_new, event_edit, event_delete
plot	INSERT, UPDATE, DELETE	plot_new, plot_edit, plot_delete
crop_catalog	INSERT, UPDATE, DELETE	catalog_add, catalog_edit, catalog_delete

If she only wants one, open the **farm_event** row: it's the harvest from the demo.

"Why did the harvest survive when you deleted the plot?"

> "The foreign key from the event to the plot says ON DELETE SET NULL. Deleting the plot blanks the plot on its events instead of deleting them."

Show: farm_event.plot_id

"What happens if you delete a whole farm?"

> "Its plots, catalog and events go with it, because those foreign keys are ON DELETE CASCADE. Those rows only mean something inside that farm."

Show: plot.farm_id

4. Search Queries

"Show me the search query."

> "The crop search looks in three places at once: the scientific name, the common name and the product titles. Then it ranks by how many auction prices each plant has, so the plant people actually grow comes first."

Show: CATALOG_SEARCH_SQL

"Show me the join."

> "This is harvest versus market. It starts from the farm's events and joins seven tables: farm, event kind, taxonomy, product, product alias and auction lots. It only counts event kinds marked as a yield, and only lots within 30 days of the harvest. It joins on the taxon, not the name, so every onion lot counts however the auction spelled it."

Show: HARVEST_PRICES_SQL
Point at: the JOIN lines 92 to 101, then the 30-day window on lines 100 and 101.

"How is the price curve calculated?"

> "It groups every price for that plant by month across all the years, and takes the lowest low, the average, and the highest high. It only uses one unit at a time, because a price per peck can't be averaged with a price per bushel."

Show: PRICE_CURVE_SQL

"Where do the numbers on the home page come from?"

> "Each one is a COUNT on its table, run live. The match percentage comes from a view."

Show: TABLE_COUNTS_SQL and v_resolution_coverage

5. Security

"How do you prevent SQL injection?"

> "Every value goes in as a bound parameter, like percent farm_id. We send Postgres the SQL text and the values separately, so whatever a user types is only ever treated as data, never run as SQL."

Show: INSERT_EVENT_SQL, then cur.execute(sql, params)

"Can one farm change another farm's records?"

> "No. Every UPDATE and DELETE has AND farm_id equals this farm in the WHERE clause. If the event ID belongs to a different farm, nothing matches and nothing changes."

Show: UPDATE_EVENT_SQL and DELETE_EVENT_SQL
Point at: line 57 and line 62.

"Is there a login?" (if asked)

> "Not yet. Accounts come with deployment the week of November 9. Each farm already has a unique owner email in the schema so we can tie a login to it."

6. The Data

"Where did your data come from?"

> "Public sources, collected by our own scripts in the loaders folder: the USDA Market News archive for Mount Hope, County Line's posted market reports, the GBIF plant taxonomy, and OpenStreetMap for Ohio farms. One script downloads all of it and another builds the database."

Show: the loaders folder

"What did you do with bad rows?"

> "Before a price goes in, we test it against the same rules as the schema. If it fails, it goes to the quarantine table with the reason, so nothing is silently dropped. 26 rows went there, like the cabbage with an average above its high."

Show: check_lot() and where it's used

"How did you stop the same farm being loaded twice?"

> "The loader keeps one farm per name and county, and keeps a web domain only once. The schema backs that up with UNIQUE on the domain."

Show: farm loading

7. Advanced Functions

There is no code for these yet. Say so plainly: they are the Stage 5 work. If she asks to see code, answer:

> "We've tested the roll-up and the ranking as queries, but the pages aren't built yet. That's this week and next week. We can show you the schema they run on."

Then open the taxon table and point at `parent_taxon_id`, the column the recursion follows.

Question	Answer
Why is the roll-up advanced?	"The number of levels under a node changes from branch to branch, so a fixed number of joins can't cover it. A recursive CTE walks any depth in one query. On Asparagales it walks 305 nodes and finds 1,805 prices."
Could the recursion loop forever?	"A CHECK stops a node being its own parent, every node has a unique path, and depth is limited to 0 through 11."
How does the decision engine score crops?	"Mean price divided by one plus how much the price swings, compared only within one unit. A crop needs at least 12 lots across two or more years. Then RANK OVER orders them."

Question	Answer
What does the ladder do with two equal matches?	"It marks the label ambiguous and stops. A person approves every match."
How will you know the ladder worked?	"The match rate on the home page goes up from 85.8 percent."

If You Don't Know

> "I'm not sure off the top of my head. Let me open the code and check."

Then open the file from the tables above. Looking it up is better than guessing.